N8n Job-Candidate Matching Engine

Automated Recruitment System Using Vector Similarity, Local Embeddings and Generative AI

Student / Developer

Dept. of Computer Engineering | University (Affiliation)

Ankara, Turkey

***Abstract*—*This paper presents an automated job-candidate matching engine built on N8n workflow automation, PostgreSQL with the pgvector extension, Ollama local embedding model (nomic-embed-text, 768 dimensions), and Google Gemini 2.5 Flash generative AI. The system ingests candidate CVs and job descriptions through webhook-triggered flows, converts them into dense vector representations, and performs cosine-similarity queries daily at 09:00 to identify matches with a similarity score of 0.75 or above. Matched candidates automatically receive an AI-generated explanatory e-mail notification. All components run in Docker containers, making the solution fully self-hosted and cost-effective.*

Keywords—n8n, pgvector, Ollama, Gemini AI, vector similarity, recruitment automation, Docker, PostgreSQL

I. Introduction

Matching job seekers with suitable positions is a time-consuming task when performed manually. Traditional keyword-based approaches fail to capture semantic relationships between a candidate's experience and a job description. This project addresses that gap by implementing a fully automated matching pipeline that leverages dense vector embeddings and cosine similarity to surface semantically relevant matches, then uses a large language model (LLM) to generate human-readable explanations.

The system is designed around three N8n workflows: (1) Candidate Registration, (2) Job Posting Registration, and (3) a scheduled Matching Engine. The entire stack runs locally inside Docker, eliminating cloud dependency for the embedding step and keeping operational costs near zero.

II. System Architecture

The system is composed of four primary technology layers as shown in Table I.

TABLE I. Technology Stack

Layer	Technology
Workflow Automation	N8n 2.18.5 (Self-Hosted, Docker)
Database	PostgreSQL 15 + pgvector extension
Embedding Model	Ollama nomic-embed-text (768-dim, local)
Generative AI	Google Gemini 2.5 Flash
Email Delivery	Gmail SMTP (App Password, port 465)
Container Runtime	Docker Desktop

Three independent N8n workflows orchestrate data ingestion and matching:

TABLE II. Workflow Summary

Workflow	Trigger & Purpose
Flow 1 - Candidate Registration	POST /register-candidate | Embeds CV and stores in candidates table
Flow 2 - Job Posting Registration	POST /register-job | Embeds job description and stores in jobs table
Flow 3 - Matching Engine	Daily Schedule 09:00 | Cosine-similarity query, Gemini call, e-mail dispatch

III. Database Design

The database contains three tables managed inside a PostgreSQL instance with the pgvector extension enabled. The embedding column type is VECTOR(768) to match the output of the nomic-embed-text model. A critical distinction from OpenAI Ada-002 (1536 dimensions) must be observed: mismatched dimensions will cause INSERT failures.

A. Schema Overview

candidates: id, name, email (UNIQUE), raw_cv, cv_summary, embedding VECTOR(768), created_at

jobs: id, company, position, description, job_summary, embedding VECTOR(768), is_active BOOLEAN, created_at

matches: id, candidate_id (FK), job_id (FK), score FLOAT, explanation TEXT, notified_at, created_at

B. Performance Indexes

IVFFlat indexes using cosine distance are created on both embedding columns to accelerate approximate nearest-neighbor search:

```
CREATE INDEX ON candidates USING ivfflat (embedding vector_cosine_ops);
CREATE INDEX ON jobs       USING ivfflat (embedding vector_cosine_ops);
```

IV. Workflow Detail

A. Candidate Registration (Flow 1)

A POST request to /register-candidate carries name, email, and cv_text fields. The Ollama Embeddings API converts the CV text into a 768-dimensional vector. A JavaScript Code node formats the data, and a PostgreSQL node inserts (or upserts on duplicate email) the record.

B. Job Posting Registration (Flow 2)

A POST request to /register-job carries company, position, and description fields. The pipeline mirrors Flow 1: Ollama embedding followed by an INSERT INTO jobs statement.

C. Matching Engine (Flow 3)

Triggered every day at 09:00 by a Schedule node, this workflow:

- Fetches all active job postings from the database.
- Iterates over each posting using Loop Over Items.

- Executes a cosine-similarity SQL query (score = 1 - cosine_distance) filtering candidates with score >= 0.75, ordered descending, limited to 10 results.
- For each match, calls Google Gemini 2.5 Flash to produce a 2-3 sentence explanation of why the candidate fits the position.
- Persists the match record to the matches table including score and AI explanation.
- Sends a personalized HTML e-mail notification to the candidate via Gmail SMTP.

The vector similarity SQL:

```
SELECT c.id AS candidate_id, c.name, c.email, c.cv_summary,
       j.id AS job_id, j.position, j.company,
       1 - (c.embedding <=> '[job_embedding]'::vector) AS score
FROM candidates c
WHERE 1 - (c.embedding <=> '[job_embedding]'::vector) >= 0.75
ORDER BY score DESC LIMIT 10;
```

V. Installation & Configuration

Prerequisites: Docker Desktop, Node.js v22 LTS (v25+ incompatible due to isolated-vm), Ollama v0.22.1+, and a Gmail account with 2-Step Verification for SMTP App Password generation.

TABLE III. Setup Steps

Step	Command / Action
1 - PostgreSQL + pgvector	docker run -d --name postgres-matching -e POSTGRES_PASSWORD=sifre123 -e POSTGRES_DB=jobmatching -p 5432:5432 ankane/pgvector
2 - N8n	docker run -it --rm --name n8n -p 5678:5678 --add-host=host.docker.internal:host-gateway -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
3 - Ollama	Install from ollama.com, then: ollama pull nomic-embed-text
4 - Gmail SMTP	myaccount.google.com/apppasswords -> create 16-char App Password
5 - Gemini API Key	aistudio.google.com -> obtain API key, insert in HTTP Request node URL
6 - Import Flows	Import workflow JSON files in N8n -> Publish each

N8n web interface: http://localhost:5678 | Ollama API: http://localhost:11434

VI. Testing

All three workflows were tested using PowerShell Invoke-RestMethod commands. Embedded vectors were verified by inspecting the first 50 characters of the stored vector representation in PostgreSQL.

The daily matching trigger was manually activated in N8n to confirm end-to-end operation including Gemini explanation generation and Gmail delivery.

TABLE IV. Test Commands

Test	Command
Register Candidate	Invoke-RestMethod -Method POST -Uri http://localhost:5678/webhook/register-candidate -ContentType application/json -Body '{"name":"Ali Yilmaz","email":"ali@test.com","cv_text":"5 yil Python..."}'
Register Job	Invoke-RestMethod -Method POST -Uri http://localhost:5678/webhook/register-job -ContentType application/json -Body '{"company":"Tech A.S.","position":"Python Dev",...}'
Verify Candidates	docker exec -it postgres-matching psql -U postgres -d jobmatching -c "SELECT id, name, email FROM candidates;"
Verify Matches	docker exec -it postgres-matching psql -U postgres -d jobmatching -c "SELECT * FROM matches;"
Test Ollama	Invoke-RestMethod -Method POST -Uri http://localhost:11434/api/embeddings -ContentType application/json -Body '{"model":"nomic-embed-text","prompt":"test"}'

VII. Deployment Status

TABLE V. Deployment Checklist

Item	Status
Node.js v22 LTS installed	DONE
PowerShell ExecutionPolicy corrected	DONE
PostgreSQL + pgvector running in Docker	DONE
Database tables created (VECTOR 768)	DONE
N8n installed and running	DONE
Ollama v0.22.1 installed	DONE
nomic-embed-text model downloaded	DONE
Gemini API key configured	DONE
Gmail SMTP App Password set	DONE
Flow 1 - Candidate Registration tested	DONE

Flow 2 - Job Posting Registration tested	DONE
Flow 3 - Matching Engine tested	DONE
Dynamic To Email in Send Email node	PENDING
Auto-generate cv_summary via Gemini	PENDING
Prevent duplicate match notifications	PENDING
Webhook authentication (Header Auth)	PENDING
Production environment (Docker Compose)	PENDING

VIII. Future Work

A. Dynamic Email Recipients

The Send Email node currently uses a hard-coded recipient. The To Email field must be set to ={{ $json.email }} (with a leading = for expression mode) and the Code node must return the email field explicitly.

B. Automated cv_summary Generation

The cv_summary field is currently NULL. Flow 1 should be extended with a Gemini summarisation step that produces a concise 2-3 sentence profile stored back in the candidates table.

C. Duplicate Match Prevention

The current INSERT INTO matches does not check for existing records. A WHERE NOT EXISTS sub-query should be added to avoid re-notifying candidates about the same position on subsequent days.

D. Webhook Security

Webhooks are currently unauthenticated. N8n's Header Auth option should be enabled (X-API-Key header) to prevent unauthorised ingestion of data.

E. Error Handling

An Error Trigger node should be added to each workflow to capture runtime failures and dispatch alert e-mails to the system administrator.

F. Production Deployment

A Docker Compose file consolidating all services, a .env file for secrets management, Docker restart policies, and automated backup of the n8n_data volume are planned for the production rollout.

IX. Conclusion

This project demonstrates a fully functional, self-hosted recruitment matching engine that combines open-source workflow automation, local embedding inference, relational vector search, and generative AI summarisation. The three-workflow architecture cleanly separates data ingestion from matching logic and notification delivery. All core functionality has been tested successfully. The remaining

roadmap items constitute straightforward incremental improvements addressable in subsequent development sprints.

Acknowledgment

The author thanks the open-source communities behind N8n, pgvector, and Ollama for providing the foundational components that made this integration possible. Google Gemini API was accessed under the free-tier quota available at aistudio.google.com.

References

[1] N8n GmbH, "n8n - Workflow Automation," https://n8n.io, 2025.

[2] A. Stucchi et al., "pgvector: Open-source vector similarity search for Postgres," https://github.com/pgvector/pgvector, 2024.

[3] Ollama Inc., "Ollama - Run Large Language Models Locally," https://ollama.com, 2025.

[4] nomic-ai, "nomic-embed-text - High-performance open embedding model," https://huggingface.co/nomic-ai/nomic-embed-text-v1, 2024.

[5] Google DeepMind, "Gemini 2.5 Flash - Generative AI API," https://aistudio.google.com, 2025.

[6] Docker Inc., "Docker Desktop Documentation," https://docs.docker.com/desktop/, 2025.